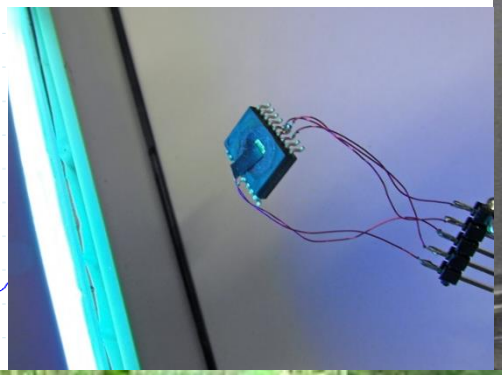
Fault injection 2

David Oswald

Code:



Goals of this lecture

After this lecture, you will …

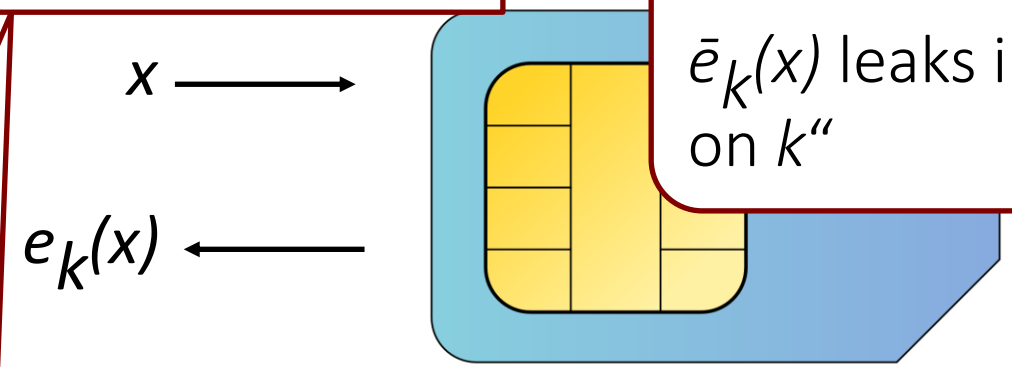
- know physical methods to inject faults from software (and some other ways)
- understand techniques to break generic code with fault injection, not just crypto

Embedded reality:

Adversa
controls

2. Control: fault injection

- Power glitch
- Clock glitch
- (Laser) light
- …



Intuition: "Faulty output $\bar{e}_k(x)$ leaks information on k"

Shortcomings:

- Physical access
- Crypto-specific

A short history of fault injection

1997: “On the importance of checking cryptographic protocols for faults” (Boneh et al.)

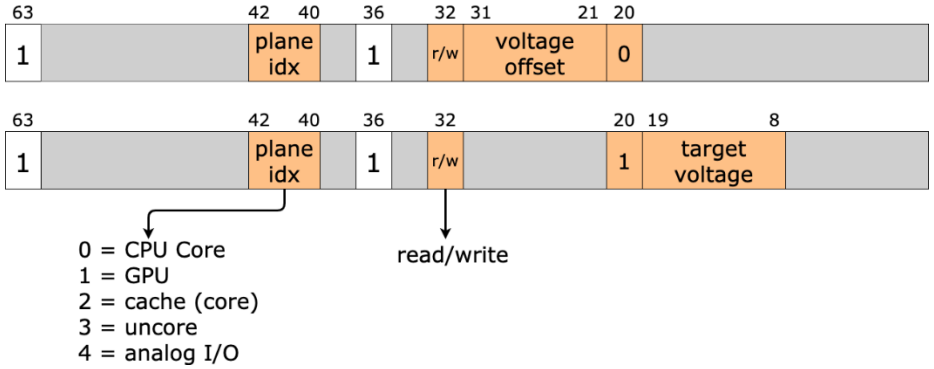
… various hardware fault attacks …

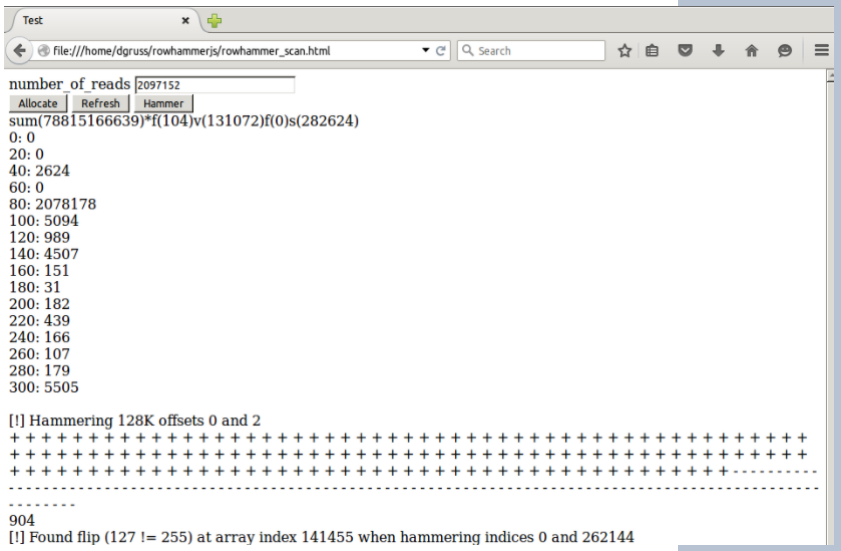
2014: “Flipping Bits in Memory Without Accessing Them: An Experimental Study of DRAM Disturbance Errors” (Kim et al.)

2015: “Exploiting the DRAM Rowhammer bug to gain kernel privileges” (Seaborn et al.)

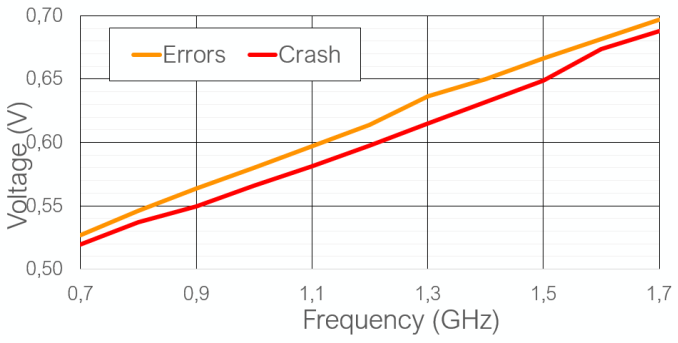
2017: “CLKSCREW: Exposing the Perils of Security Oblivious Energy Management” (Tang et al.)

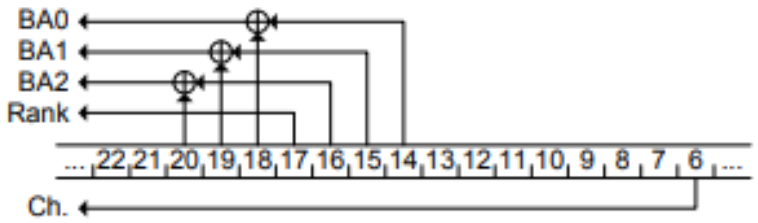
… many more software fault attacks …





Software-based fault injection

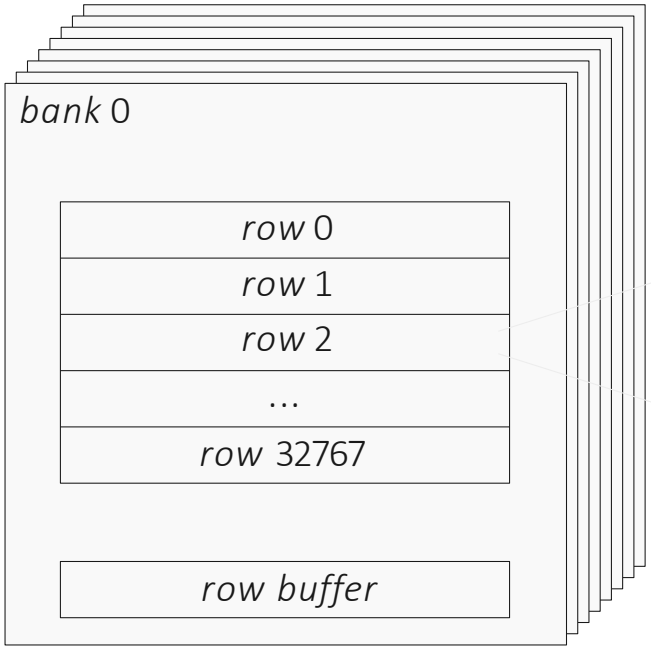




Software-based fault injection

- Physical attackers are a realistic threat for smartcards, but not for general-purpose PCs
- Assume an attacker has “just” code execution: what can we do?
- Idea: “magic” instruction sequences (that can be run in user mode): The **Rowhammer** issue

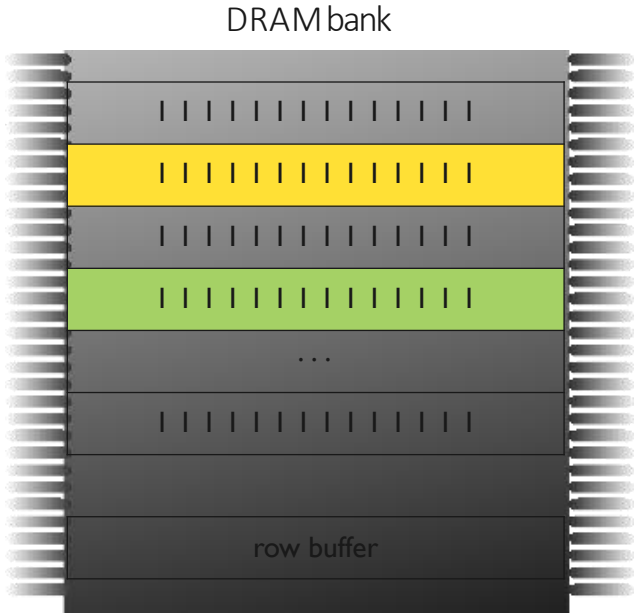
Rowhammer: DRAM organisation



64k cells
1 capacitor,
1 transistor each

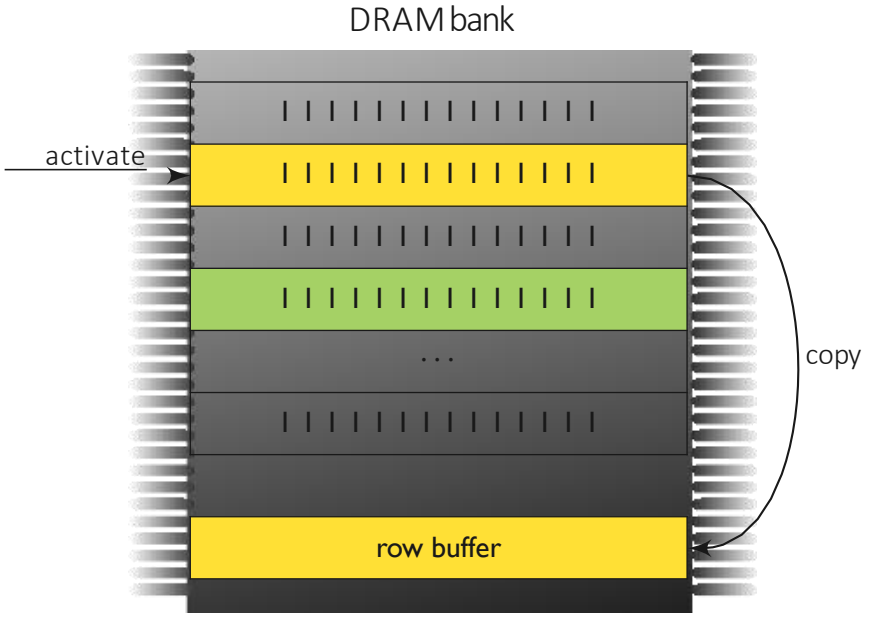
Slides: Thanks to Daniel Gruss

The Rowhammer effect



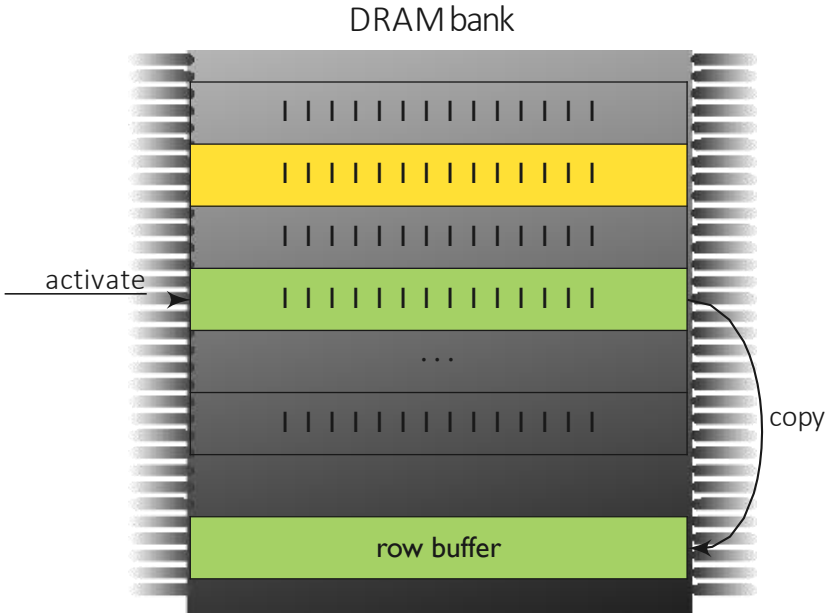
- Cells leak → need refresh
- Max. refresh interval to guarantee data integrity
- Cells leak faster upon proximate accesses → Rowhammer

The Rowhammer effect



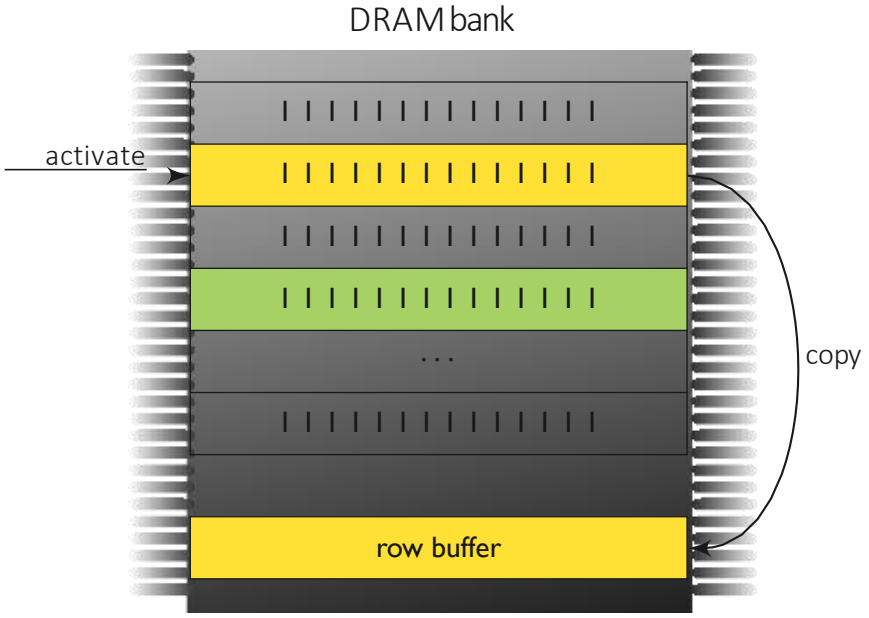
- Cells leak → need refresh
- Max. refresh interval to guarantee data integrity
- Cells leak faster upon proximate accesses → Rowhammer

The Rowhammer effect



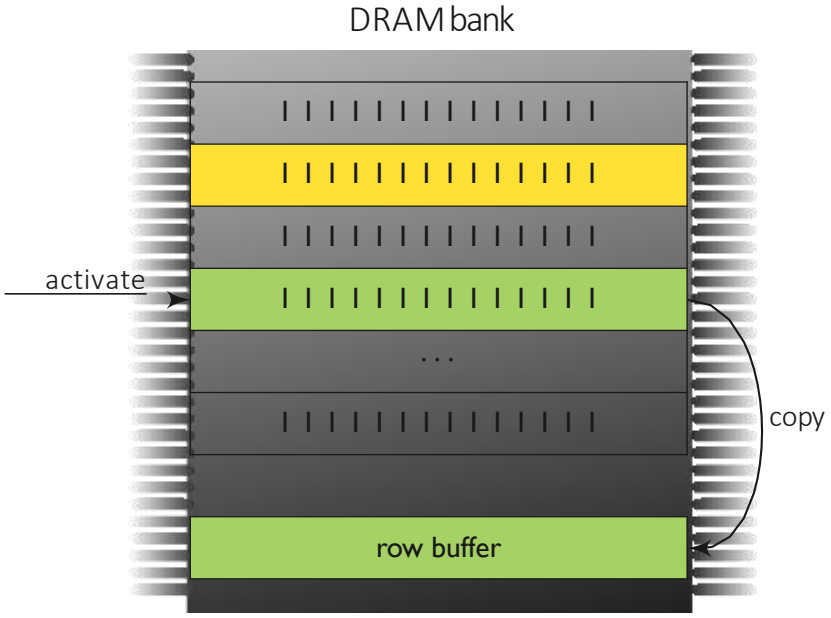
- Cells leak → need refresh
- Max. refresh interval to guarantee data integrity
- Cells leak faster upon proximate accesses → Rowhammer

The Rowhammer effect



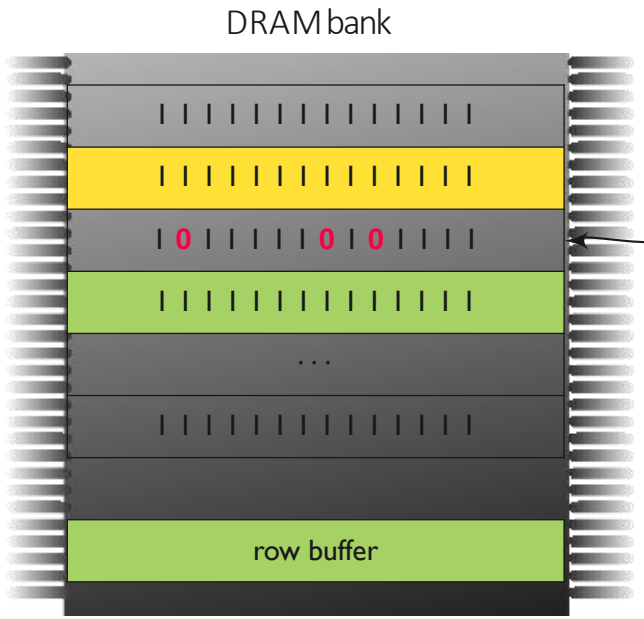
- Cells leak → need refresh
- Max. refresh interval to guarantee data integrity
- Cells leak faster upon proximate accesses → Rowhammer

The Rowhammer effect



- Cells leak → need refresh
- Max. refresh interval to guarantee data integrity
- Cells leak faster upon proximate accesses → Rowhammer

The Rowhammer effect



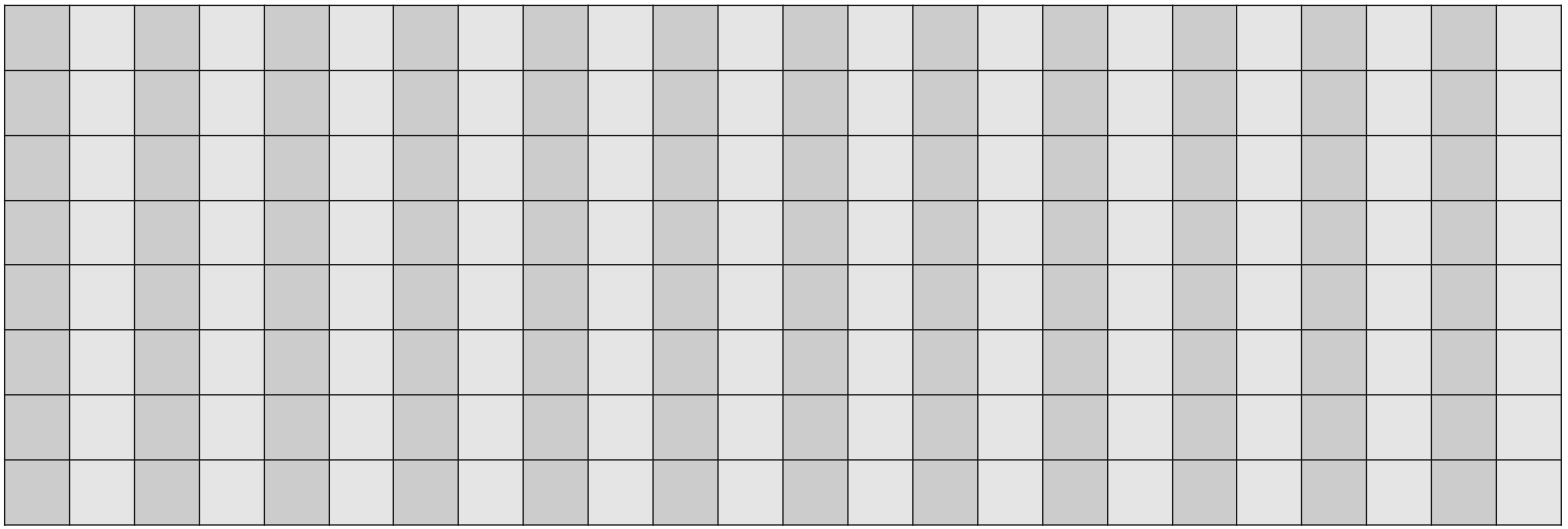
- Cells leak → need refresh
- Max. refresh interval to guarantee data integrity
- Cells leak faster upon proximate accesses → Rowhammer

Rowhammer prerequisites

We need:

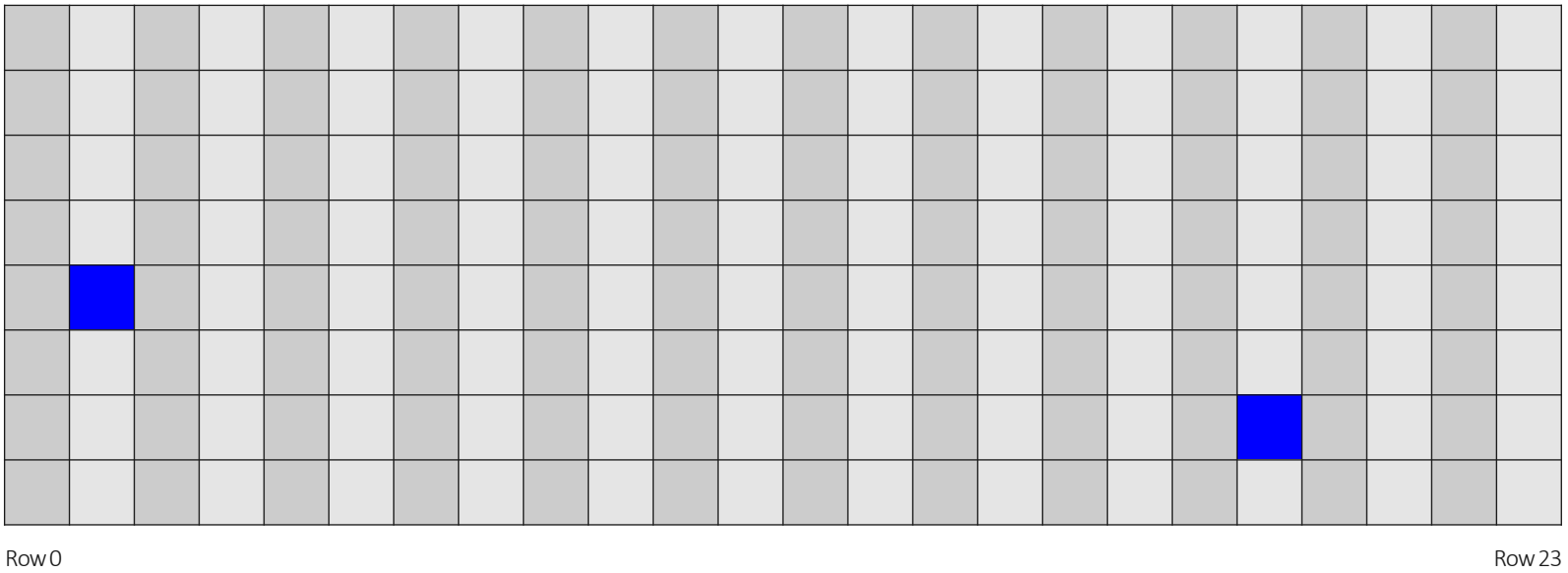
1. Uncached memory access
   → various techniques e.g. `clflush`
2. Fast memory access
   → possible
3. Target specific row
   → reverse-engineer
   mapping functions

Searching for page with flip



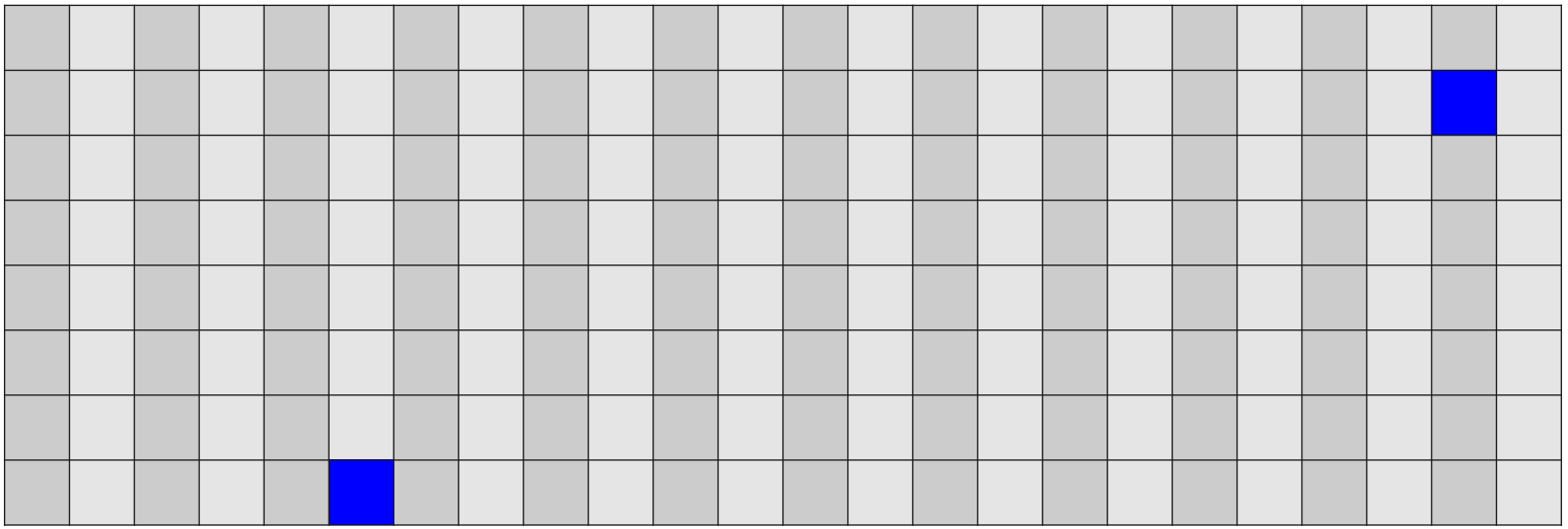
Hammering memory locations in different rows

Searching for page with flip



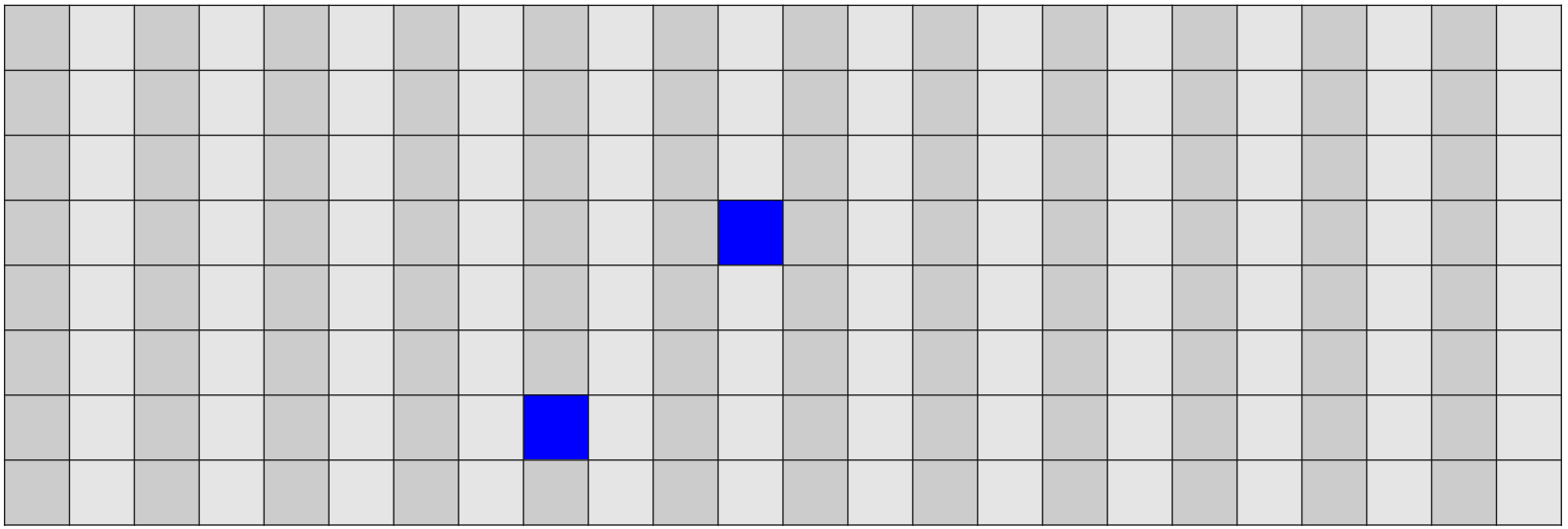
Hammering memory locations in different rows

Searching for page with flip



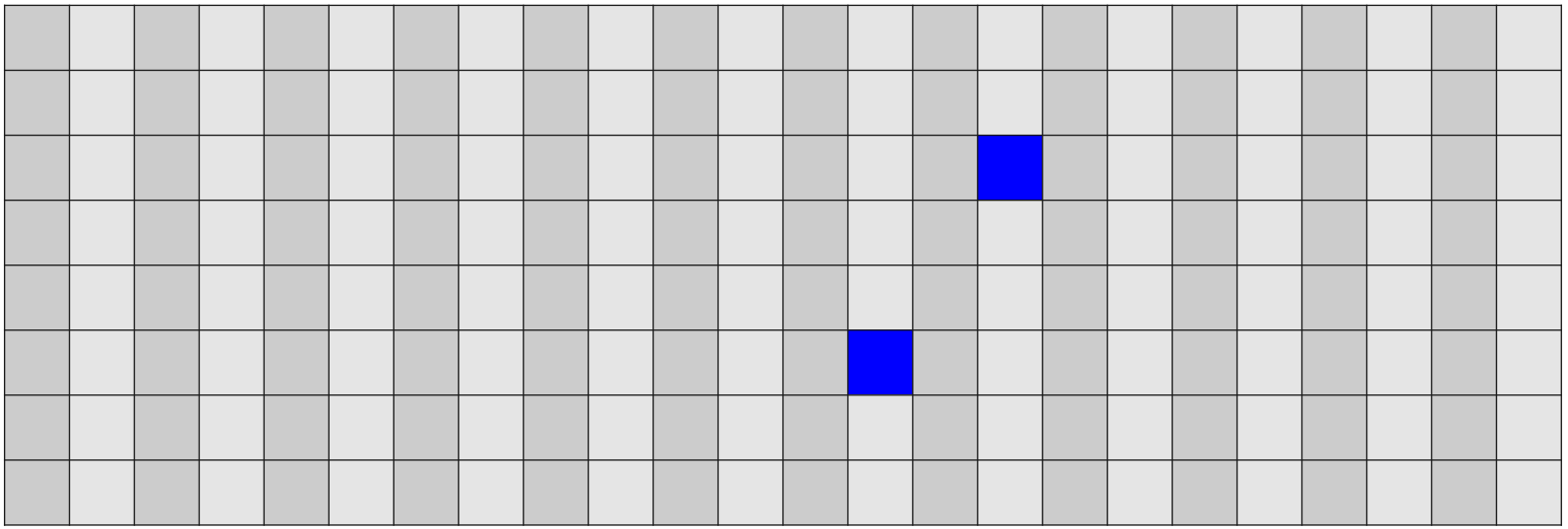
Hammering memory locations in different rows

Searching for page with flip



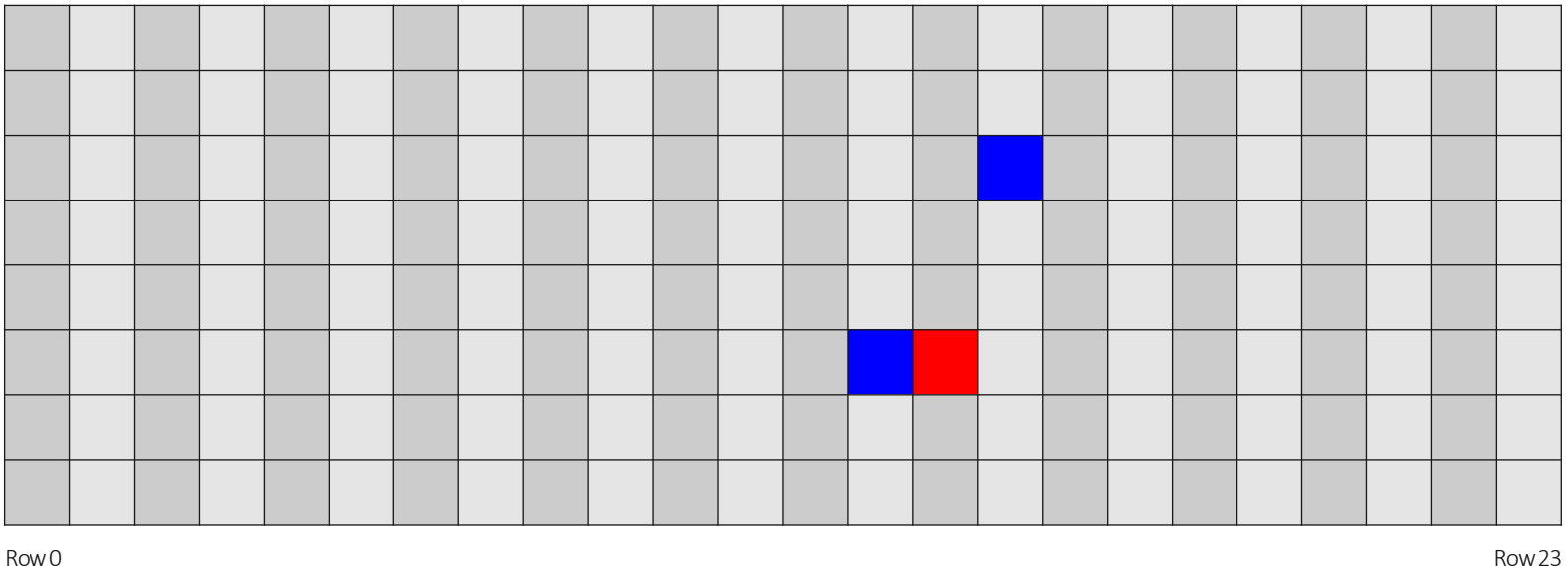
Hammering memory locations in different rows

Searching for page with flip



Hammering memory locations in different rows

Searching for page with flip



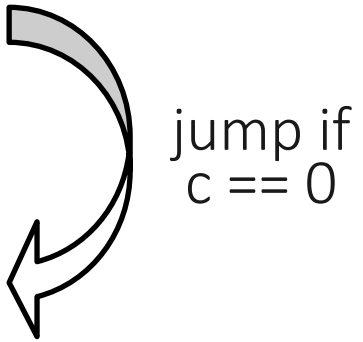
Hammering memory locations in different rows

- Found a flip

Bypassing PIN check with Rowhammer

```
int c = memcmp(entered_pin, stored_pin, pin_len);

if(c != 0)
{
    return -1;
}

// Code continues ...
```

```
cmp rax, 0
je continue
…
ret
continue:
…
```



Changing code with Rowhammer

Changing code with Rowhammer

Changing code with Rowhammer

Changing code with Rowhammer

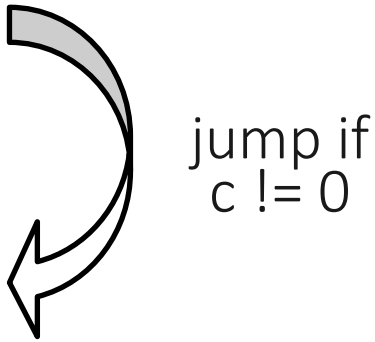
Changing code with Rowhammer

Changing code with Rowhammer

Changing code with Rowhammer

Changing code with Rowhammer

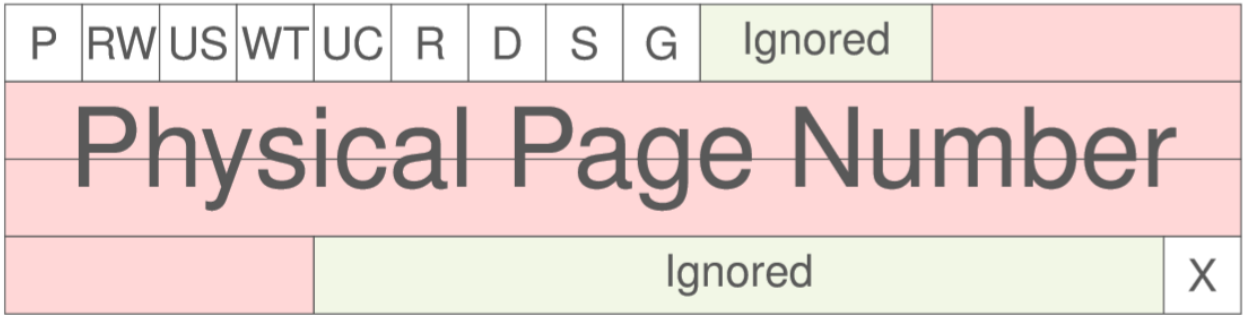

```
cmp rax, 0
jne continue
…
ret
continue:
…
```



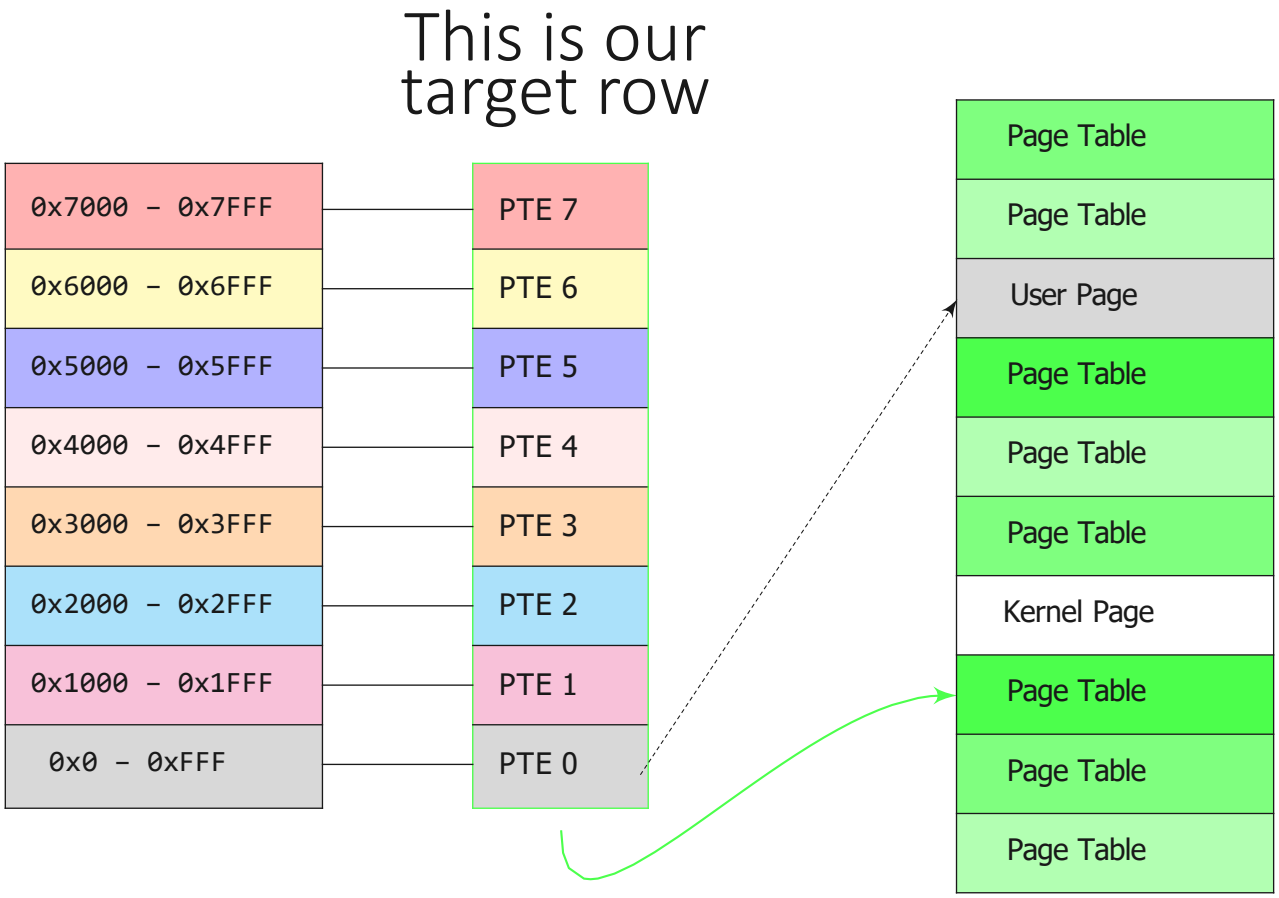
jump if
c != 0

Rowhammer on page tables

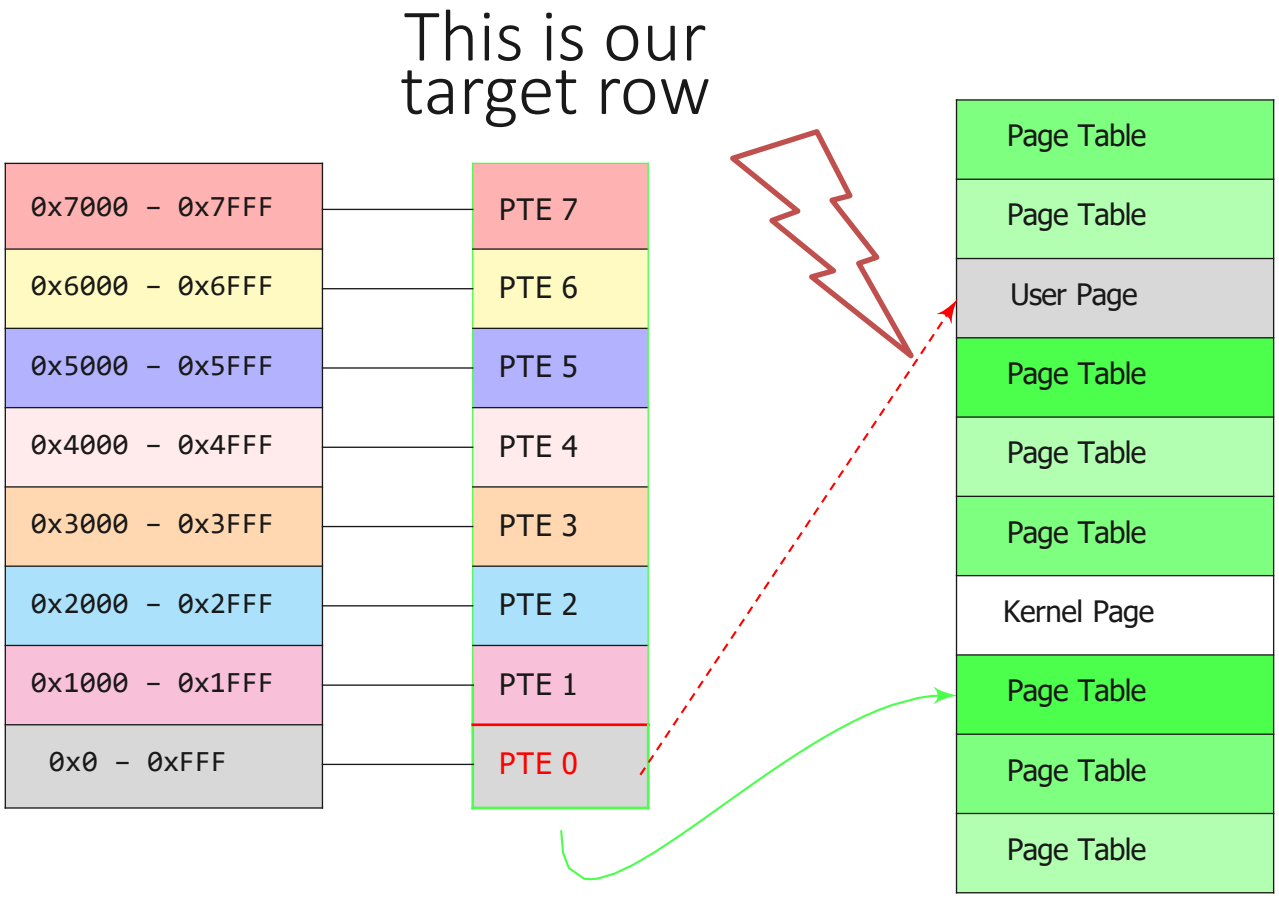
- Find a target location
- Make a page table appear at target location
- Fill physical memory with page tables
- Flip in target PTE likely gives access to a page table → access to physical memory



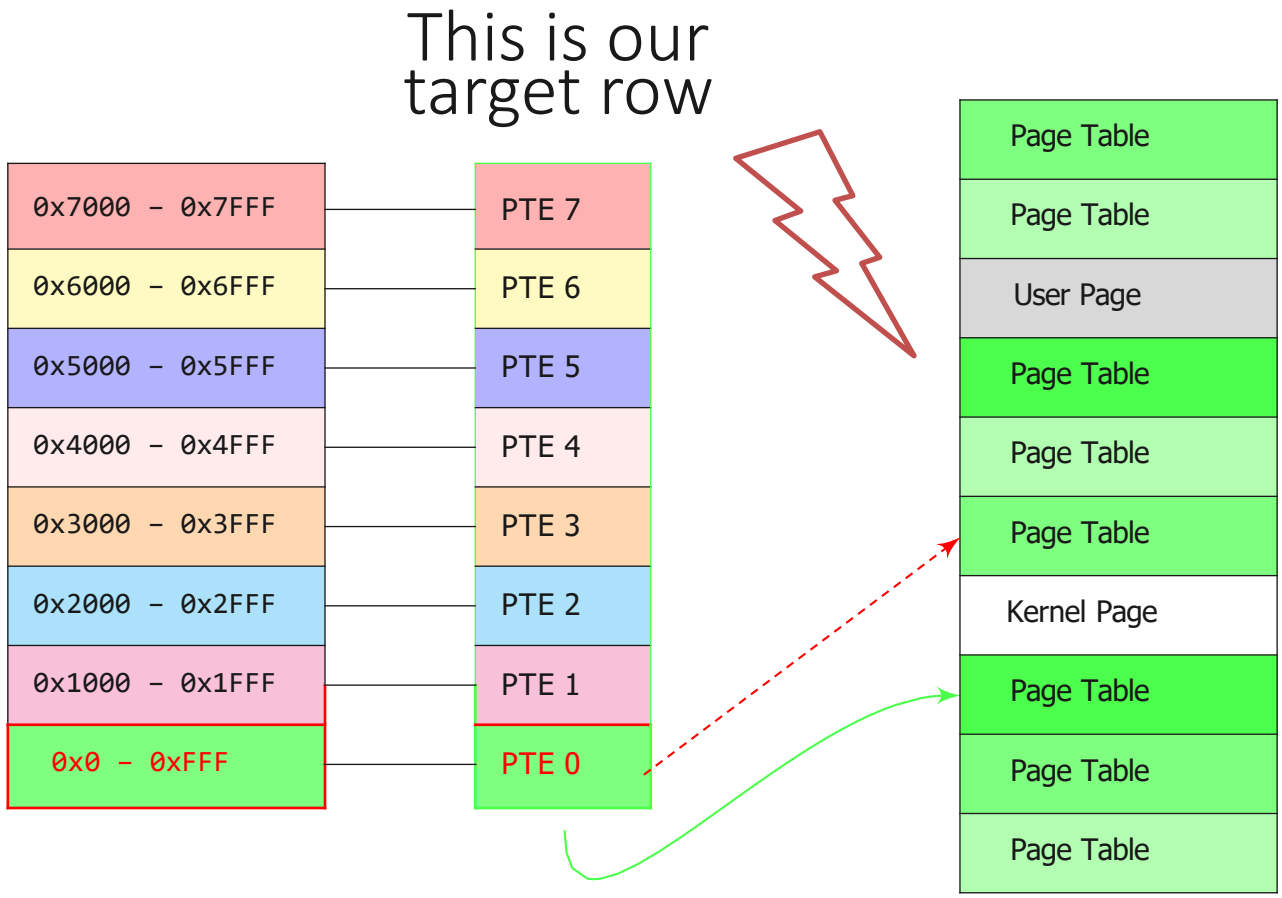
Page table example: before hammering



Page table example: flipping a PTE



Page table example: changing a PTE

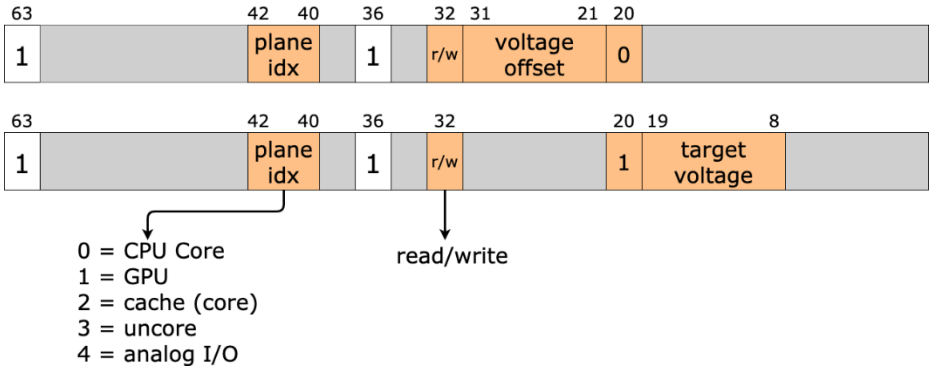


Rowhammer …

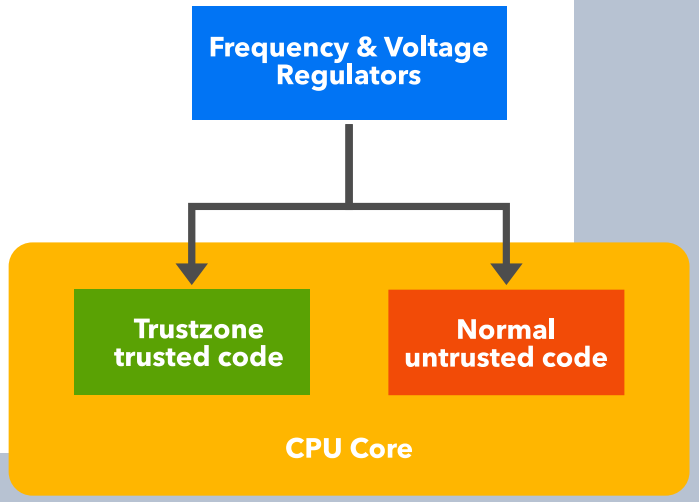
- … can be exploited in various ways
- … works from the browser (Rowhammer.js)

Rowhammer …

- … can be exploited in various ways
- … works from the browser (Rowhammer.js)
- … works on ARM smartphones (Drammer)
- … works for protected DDR4 (TRRespass)
- … can even work on flash memory (SSD):
  https://www.usenix.org/system/files/conference/woot17/woot17-paper-kurmus.pdf
- There are countermeasures, but
  they are not 100% effective



Software-based fault attacks with power management



Power and clock management

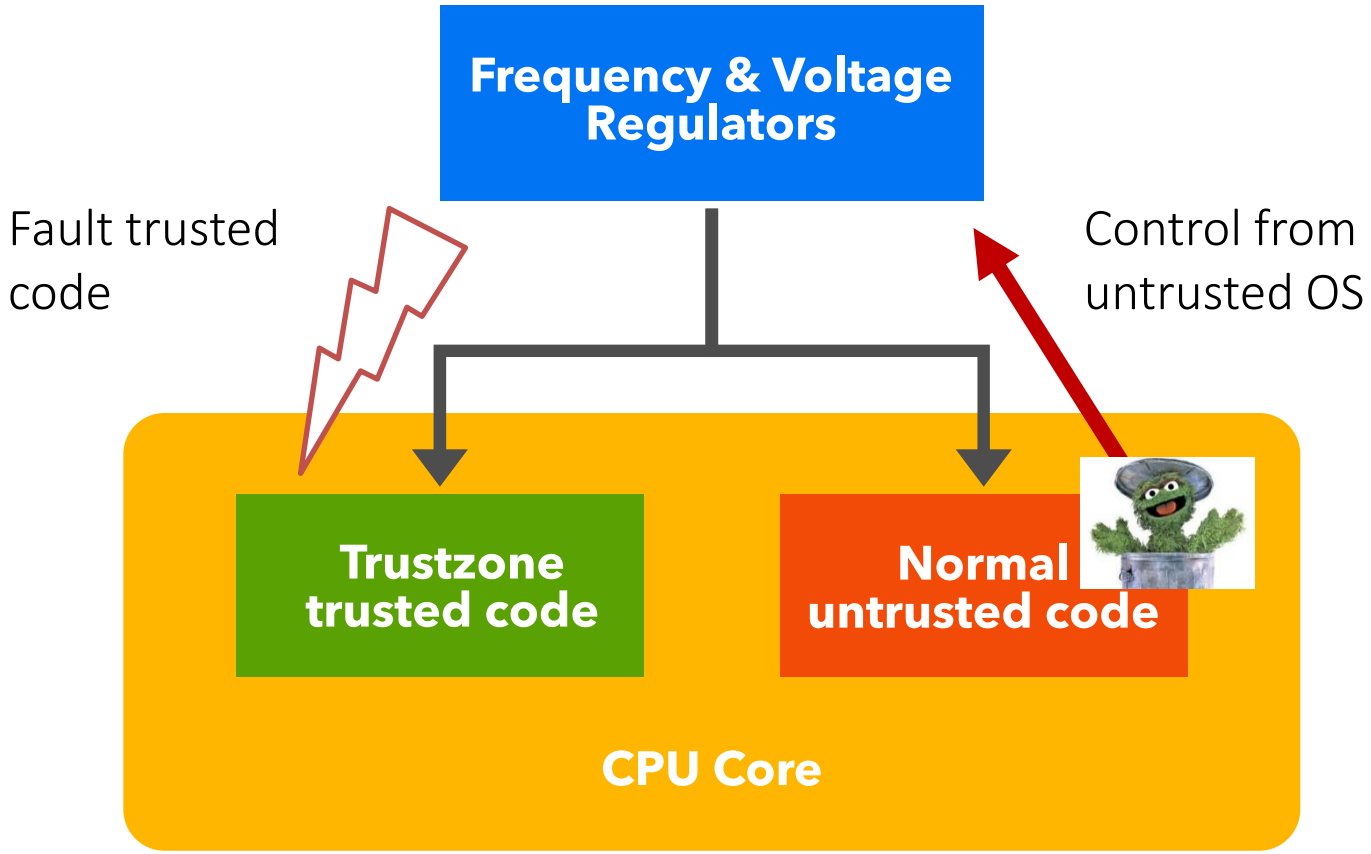
- Modern CPUs continuously adjust their clock frequency and supply voltage to run at minimum power consumption / temperature
- Dynamic Voltage and Frequency Scaling (**DVFS**)
- Too high frequency and/or too low voltage results in faults (typically crashes)
- DVFS is often software-exposed…

CLKscrew and VoltJockey on ARM

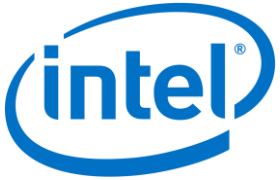
- Smartphones (based on ARM processors) allow clock and voltage control if you're **`root`**
- This can be used to inject controlled faults from privileged software:
  - Tang et al. "CLKSCREW: exposing the perils of security-oblivious energy management", USENIX Security '17
  - Qiu et al. "VoltJockey: Breaching TrustZone by Software-Controlled Voltage Manipulation over Multi-core Frequencies", CCS 2019
- Why do attacks requiring **`root`** matter?

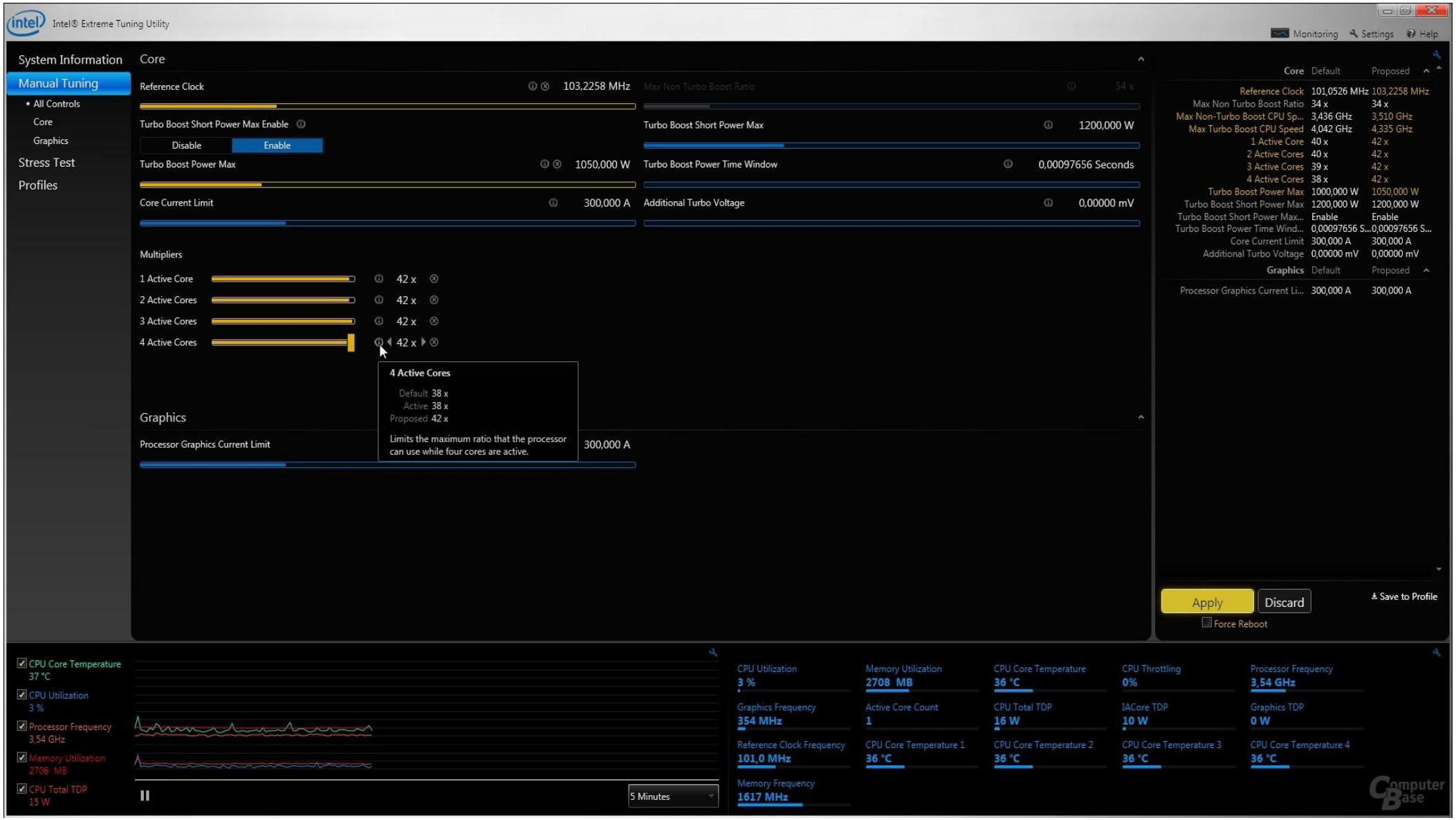
Trusted Execution Environments (TEEs)

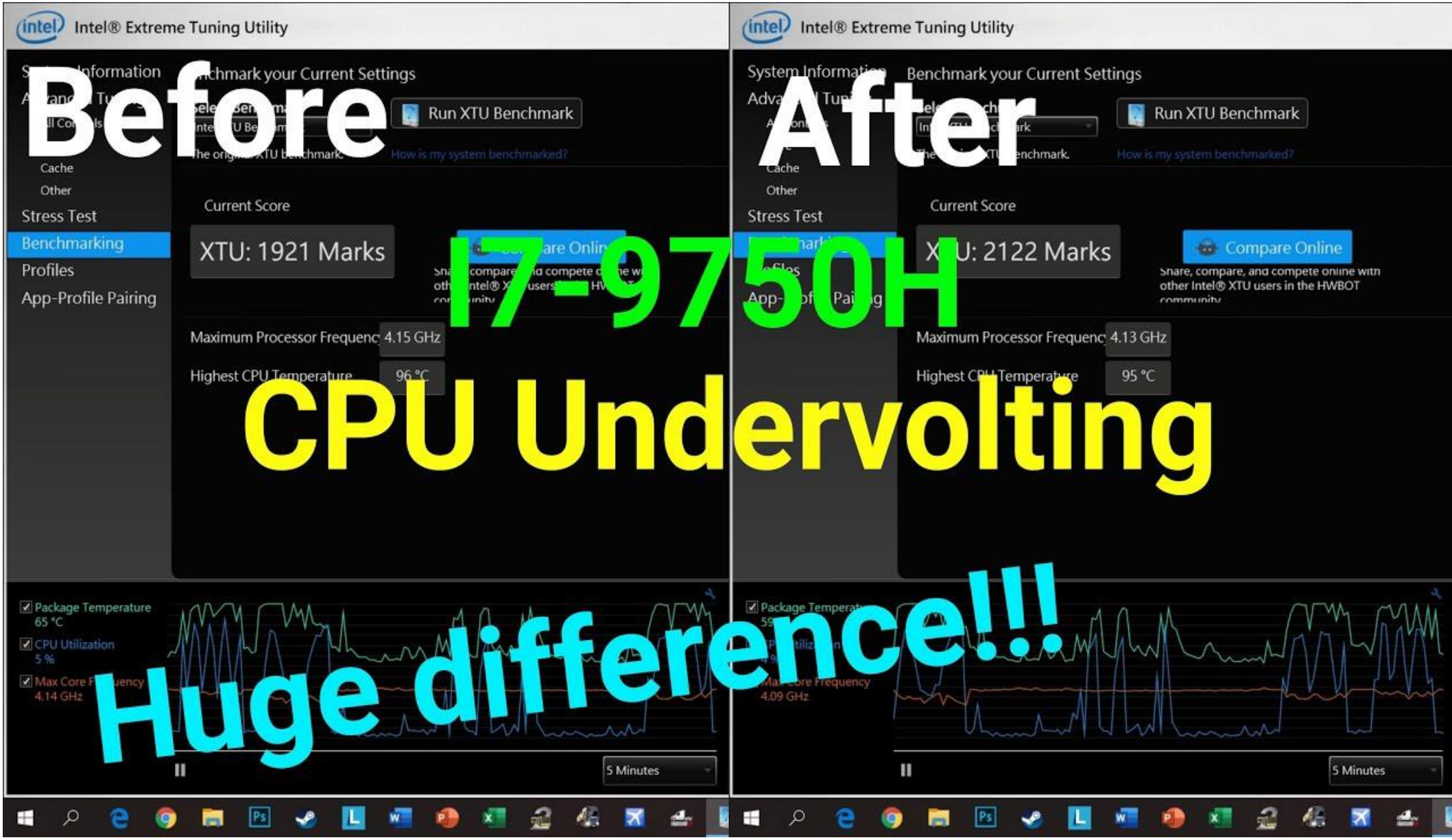
TEEs like ARM Trustzone should protect against attackers with `root` on untrusted OS



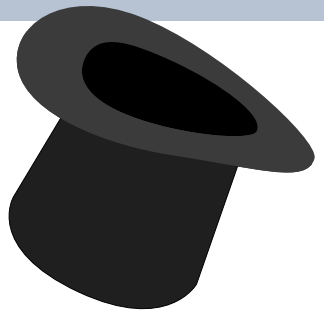
Power management attacks on Trustzone

- Attacker controls untrusted OS …
- … but manages to fault the trusted code in the TEE
- Demonstrated attacks:
  - Break AES (see previous lecture)
  - Load arbitrary code into the Trustzone by faulting a signature check
- What about 

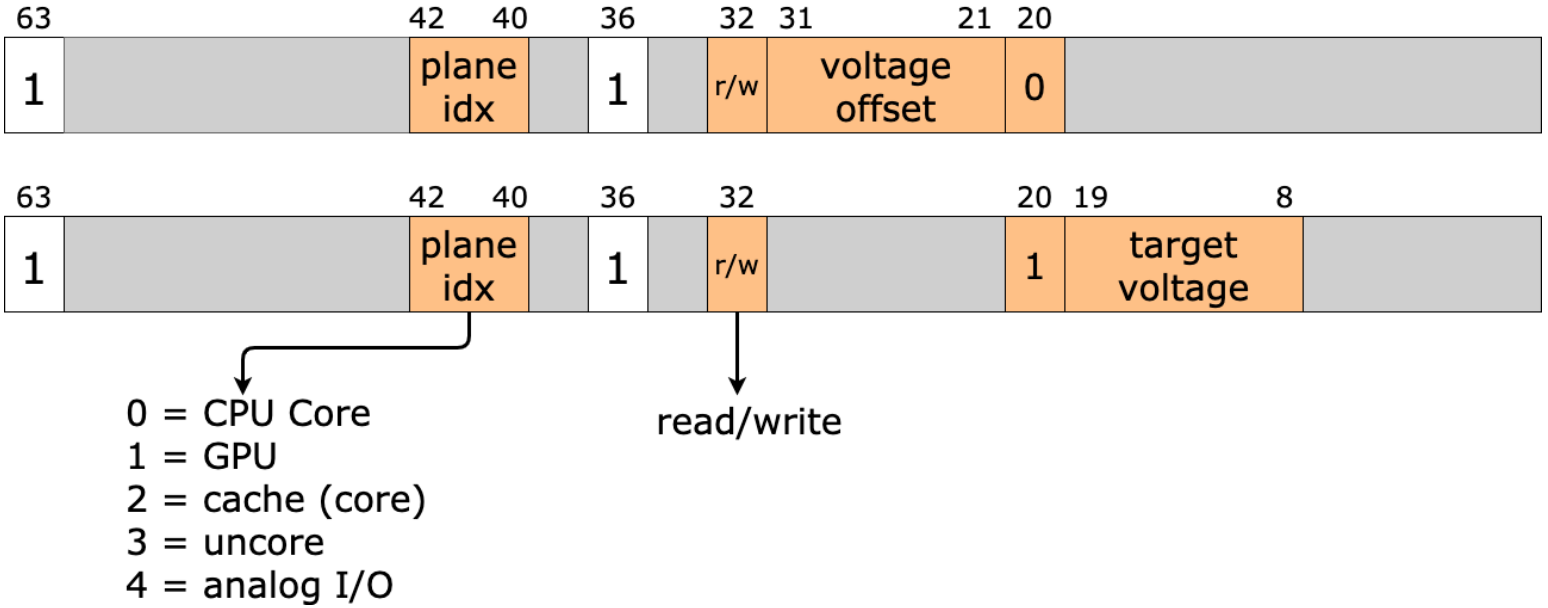

Intel® Extreme Tuning Utility
Monitoring
Settings
Help
System Information
Manual Tuning
• All Controls
Core
Graphics
Stress Test
Profiles
Core
Reference Clock 103,2258 MHz
Max Non Turbo Boost Ratio 34 x
Turbo Boost Short Power Max Enable
Disable
Enable
Turbo Boost Short Power Max 1200,000 W
Turbo Boost Power Max 1050,000 W
Turbo Boost Power Time Window 0,00097656 Seconds
Core Current Limit 300,000 A
Additional Turbo Voltage 0,00000 mV
Multipliers
1 Active Core 42 x
2 Active Cores 42 x
3 Active Cores 42 x
4 Active Cores 42 x
4 Active Cores
Default 38 x
Active 38 x
Proposed 42 x
Limits the maximum ratio that the processor can use while four cores are active.
Graphics
Processor Graphics Current Limit 300,000 A
Core Default Proposed
Reference Clock 101,0526 MHz 103,2258 MHz
Max Non Turbo Boost Ratio 34 x 34 x
Max Non-Turbo Boost CPU Sp... 3,436 GHz 3,510 GHz
Max Turbo Boost CPU Speed 4,042 GHz 4,335 GHz
1 Active Core 40 x 42 x
2 Active Cores 40 x 42 x
3 Active Cores 39 x 42 x
4 Active Cores 38 x 42 x
Turbo Boost Power Max 1000,000 W 1050,000 W
Turbo Boost Short Power Max 1200,000 W 1200,000 W
Turbo Boost Short Power Max... Enable Enable
Turbo Boost Power Time Wind... 0,00097656 S... 0,00097656 S...
Core Current Limit 300,000 A 300,000 A
Additional Turbo Voltage 0,00000 mV 0,00000 mV
Graphics Default Proposed
Processor Graphics Current Li... 300,000 A 300,000 A
Apply
Discard
Save to Profile
Force Reboot
CPU Core Temperature 37 °C
CPU Utilization 3 %
Processor Frequency 3,54 GHz
Memory Utilization 2708 MB
CPU Total TDP 15 W
5 Minutes
CPU Utilization 3 %
Memory Utilization 2708 MB
CPU Core Temperature 36 °C
CPU Throttling 0%
Processor Frequency 3,54 GHz
Graphics Frequency 354 MHz
Active Core Count 1
CPU Total TDP 16 W
IACore TDP 10 W
Graphics TDP 0 W
Reference Clock Frequency 101,0 MHz
CPU Core Temperature 1 36 °C
CPU Core Temperature 2 36 °C
CPU Core Temperature 3 36 °C
CPU Core Temperature 4 36 °C
Memory Frequency 1617 MHz
Computer Base


Before
After
i7-9750H
CPU Undervolting
Huge difference!!!
Intel® Extreme Tuning Utility
Benchmark your Current Settings
Run XTU Benchmark
How is my system benchmarked?
Cache
Other
Stress Test
Benchmarking
Profiles
App-Profile Pairing
Current Score
XTU: 1921 Marks
Maximum Processor Frequenc 4.15 GHz
Highest CPU Temperature 96 °C
Compare Online
Package Temperature
65 °C
CPU Utilization
5 %
4.14 GHz
5 Minutes
XTU: 2122 Marks
Maximum Processor Frequenc 4.13 GHz
Highest CPU Temperature 95 °C
Share, compare, and compete online with other Intel® XTU users in the HWBOT
4.09 GHz

How does that work?

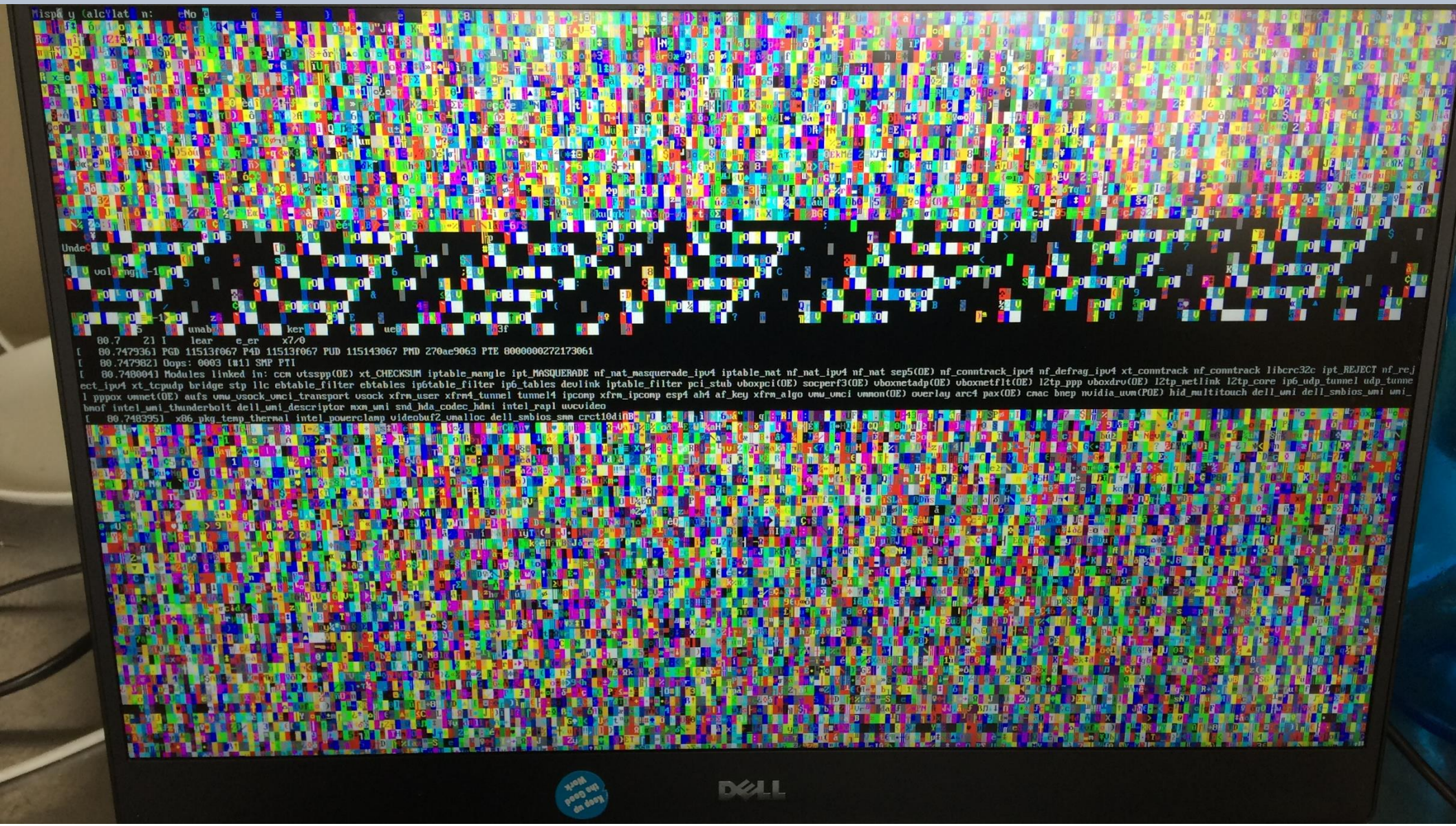


`msr 0x150`



A simple proof-of-concept

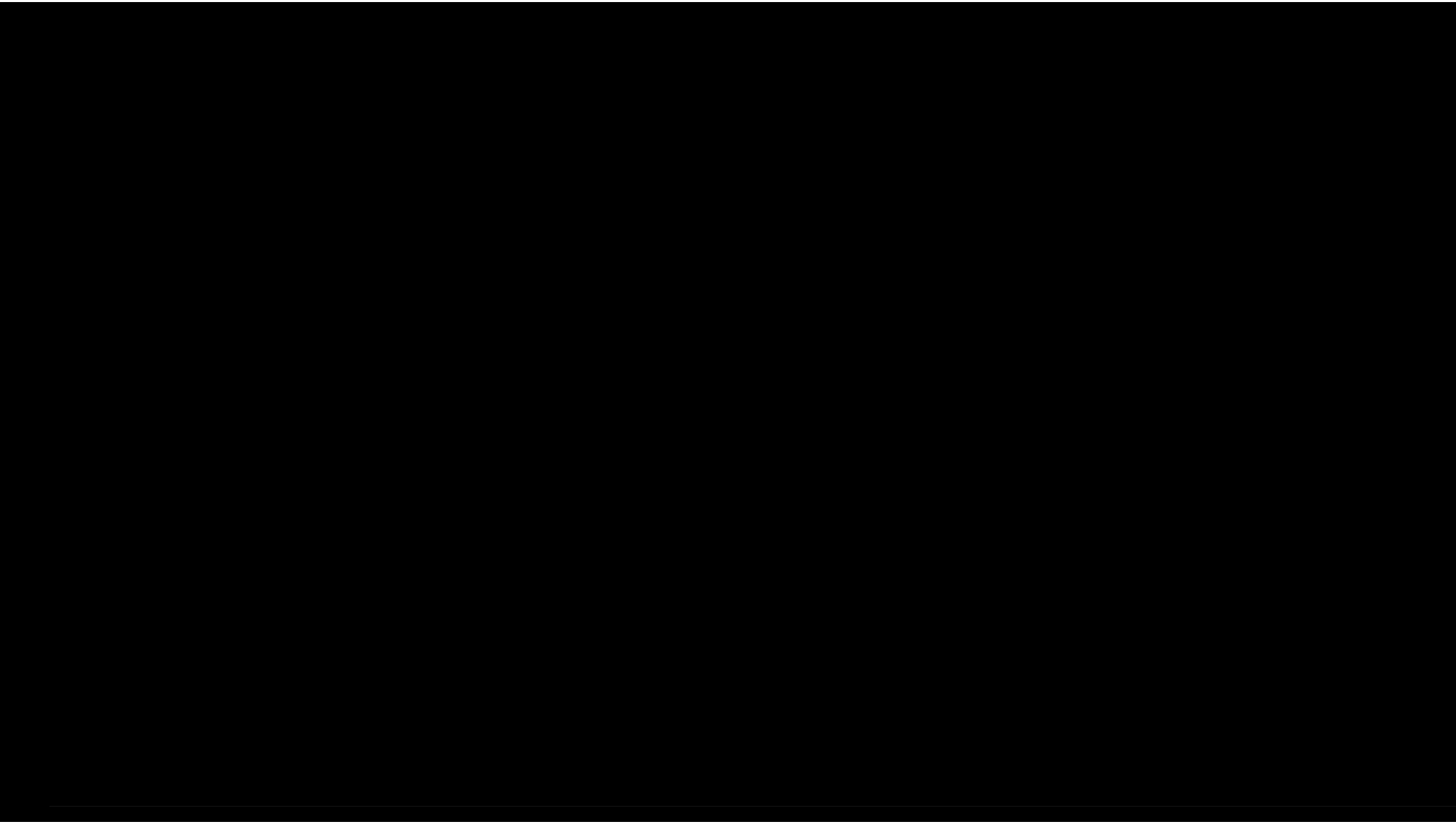
```
uint64_t multiplier = 0x1122334455667788;
uint64_t correct    = 0xdeadbeef * multiplier;
uint64_t var        = 0xdeadbeef * multiplier;

// start undervolting
while ( var == correct )
{
    var = 0xdeadbeef * multiplier;
}
// stop undervolting

// Can we ever get here? Yes!!
uint64_t flipped_bits = var ^ correct;
```


[80.7479361 PGD 11513f067 P4D 11513f067 PUD 115143067 PMD 270ae9063 PTE 8000000272173061
[80.7479821 Oops: 0003 [#1] SMP PTI
[80.7480041 Modules linked in: ccm vtsspp(OE) xt_CHECKSUM iptable_mangle ipt_MASQUERADE nf_nat_masquerade_ipv4 iptable_nat nf_nat_ipv4 nf_nat sep5(OE) nf_conntrack_ipv4 nf_defrag_ipv4 xt_conntrack nf_conntrack libcrc32c ipt_REJECT nf_reject_ipv4 xt_tcpudp bridge stp llc ebtable_filter ebtables ip6table_filter ip6_tables devlink iptable_filter pci_stub vboxpci(OE) socperf3(OE) vboxnetadp(OE) vboxnetflt(OE) l2tp_ppp vboxdrv(OE) l2tp_netlink l2tp_core ip6_udp_tunnel udp_tunnel pppox vmnet(OE) aufs vmw_vsock_vmci_transport vsock xfrm_user xfrm4_tunnel tunnel4 ipcomp xfrm_ipcomp esp4 ah4 af_key xfrm_algo vmw_vmci vmmon(OE) overlay arc4 pax(OE) cmac bnep nvidia_uvm(POE) hid_multitouch dell_wmi dell_smbios_wmi wmi_bmof intel_wmi_thunderbolt dell_wmi_descriptor mxm_wmi snd_hda_codec_hdmi intel_rapl uvcvideo
[80.7483951 x86_pkg_temp_thermal intel_powerclamp videobuf2_vmalloc dell_smbios_smm crct10dif
DELL

Live-Demo

"Anything that can go wrong, will go wrong"

Plundervolt: Multiplications



Carmen Crincoli, but Fhqwhgads
@CarmenCrincoli

"Plundervolt" sure does have a catchy name and
an exploit that
running

1:59 AM · D

Larry Osterman
@osterman

Replying to @CarmenCrincoli and @MT6572A

Since the threat model for SGX assumes that the attacker has root access (and I believe also has physical control over the hardware), this is actually a bigger deal than you make it out to be.

2:07 AM · Dec 11, 2019 · Twitter Web App

Some attacks

- Key extraction attack on RSA and AES-NI

Faulting CRT-RSA

```
// Start undervolting
uint8_t rsa_dec_ecall(int iterations)
{
    //Waitforfirstfault
    trigger_fault(iterations);

    //Actualdecryption
    ippsRSA_Decrypt(ct,dec,pPrv,scratchBuffer);
}
// Stop undervolting
```

Faulting CRT-RSA

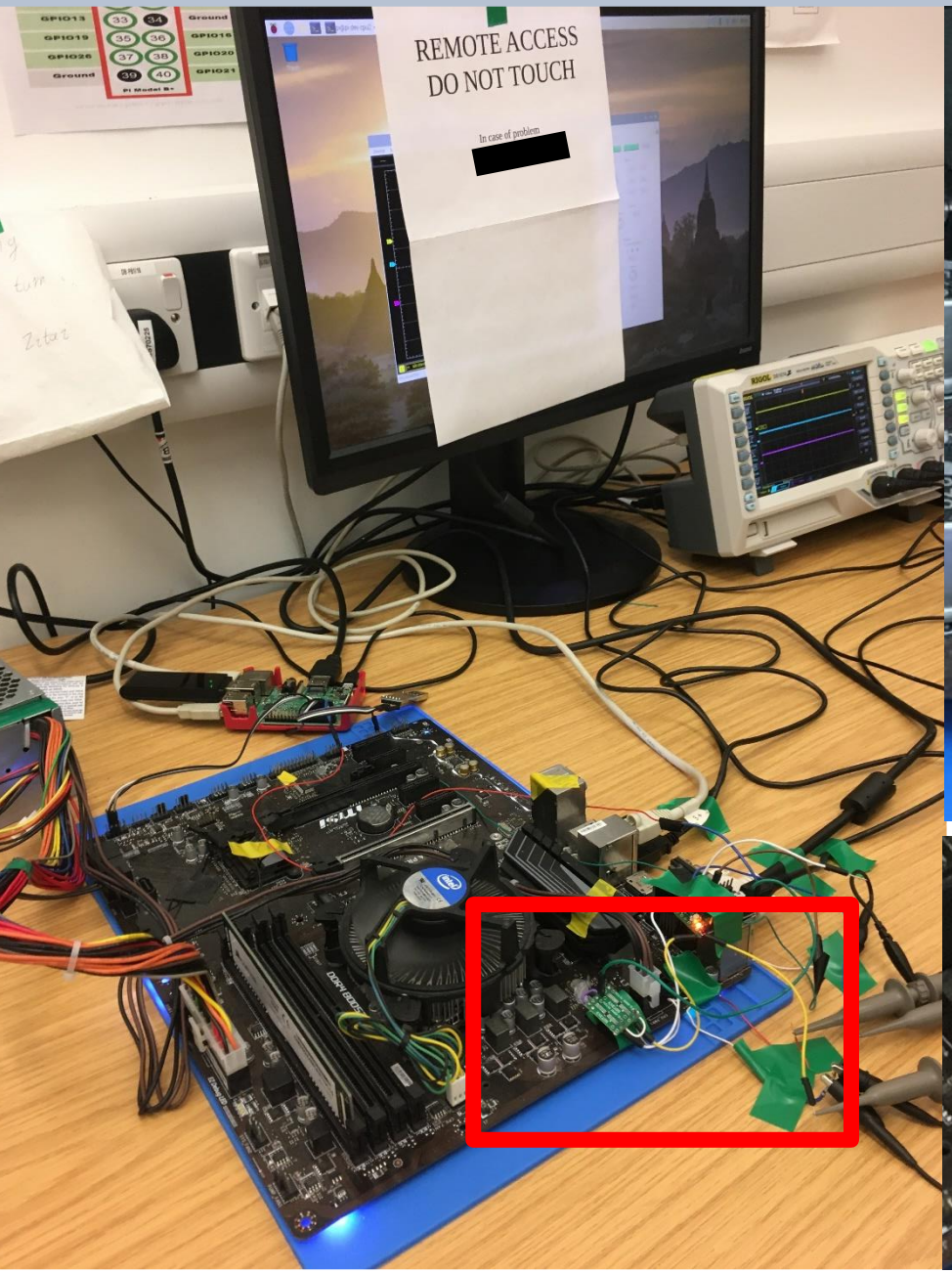
```
// Start undervolting
uint8_t rsa_dec_ecall(int iterations)
{
    //Waitforfirstfault
    trigger_fault(iterations);

    //Actualdecryption
    ippsRSA_Decrypt(ct,dec,pPrv,scratchBuffer);
}
// Stop undervolting
```

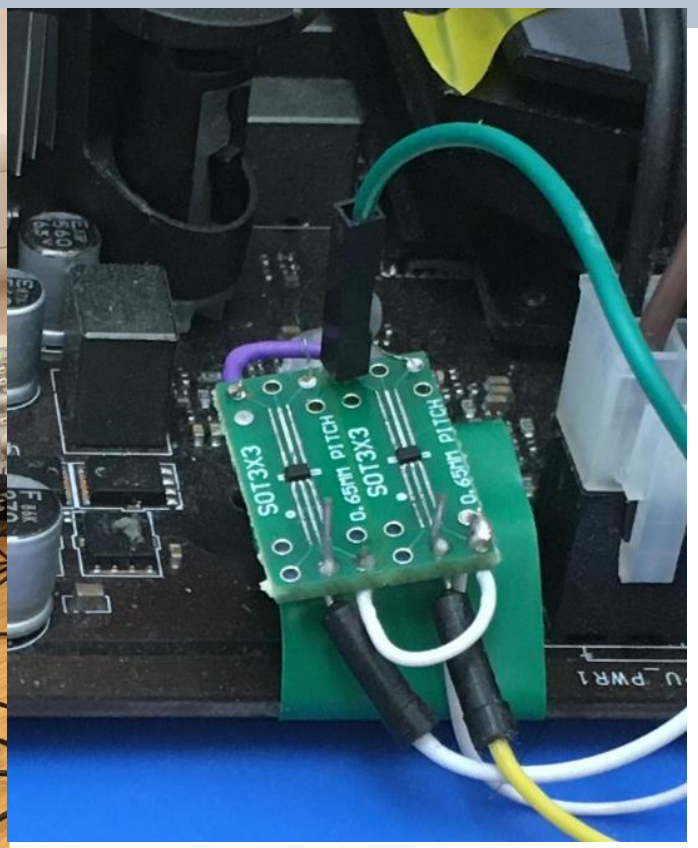
- Bellcore attack: gcd($M' - M$, n)
- Lenstra attack: gcd($(M')^e - C$, n)
- yields p or q (and dividing n by it gives the other)

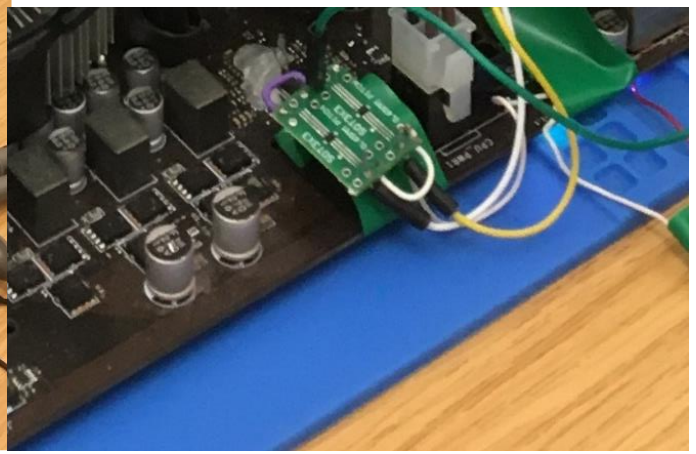
Some attacks

- Key extraction attack on RSA and AES-NI
- Induce memory safety vulnerabilities in bug-free enclave code
- Responsible disclosure: Intel released microcode update with fix for CVE-2019-11157 in Dec 2019
- Fix: turn off software undervolting interface in MSR 0x150
  (what about hardware attacks?)

Hardware-level undervolting against SGX







Take-home points

- Fault injection is not limited to hardware attackers: faults can be injected remotely
- Fault injection is not limited to crypto code: generic code can be broken (e.g. regarding memory safety)
- Countermeasures against this are even harder: PCs are general purpose systems, not high-security smartcards
- Fault injection is there to stay

Thanks!

Questions / discussion?

d.f.oswald@bham.ac.uk